



P.I. Redes / Oper

Proyecto: **Etapa #3**

Equipo: Los Padawans

Juan González Márquez - C4F588

Jeferson Marín Porras - C24549

Ariadna Ardila Gutiérrez - C4C646

Juan Sebastián Loazia Ospina B74200





PROTOCOLO

Formato del mensaje

- Todos los mensajes siguen esta estructura:
- P/Comando/Mensaje
- P: indica que se está usando este protocolo.
- Comando: letra mayúscula que indica la acción.
- Mensaje: datos enviados, como figura, piezas, IP o puerto.
- Ejemplos:
- P/G/dragon
- Significa solicitar la figura dragon.

Comandos principales

- C — Connect: conexión inicial entre Fork y servidor.
- S — Server Data: IP y puerto del servidor.
- R — Request: pedir directorio de figuras.
- G — Get: solicitar una figura.
- D — Data: respuesta con figuras o piezas.
- Q — Quit: cerrar conexión.
- F — Fork: comunicación entre Forks.
- K — Kill: servidor avisa que se desconecta.
- P — Patch: actualización de servidores



Descubrimiento de servidores

- Primero se levanta el Fork.
- El Fork envía un mensaje por broadcast preguntando por servidores disponibles:
- Fork > P/C/172.16.123.70:8080 > Broadcast
- El servidor escucha el broadcast.
- Cuando recibe el mensaje, se conecta por TCP al Fork.
- Luego envía sus datos:
- Server > P/S/172.16.123.71:8080 > Fork
- El Fork guarda ese servidor en su mapa de servidores.

Flujo cliente-Fork-servidor

- El cliente pide el directorio de figuras:
- Client > P/R/dir > Fork
- El Fork consulta a los servidores:
- Fork > P/R/dir > Server
- El servidor responde con figuras disponibles:
- Server > P/D/fig1,fig2,fig3 > Fork
- Si el cliente pide una figura:
- Client > P/G/fig1 > Fork
- El Fork busca en su mapa y consulta al servidor correcto.
- El servidor responde con las piezas y el Fork se las envía al cliente.



Actualizaciones, caídas y errores

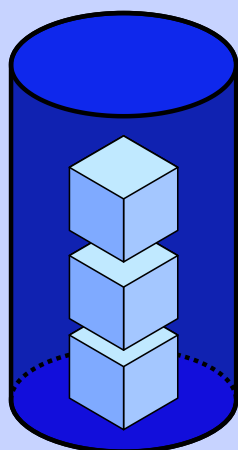
- Los Forks pueden comunicarse entre sí usando F.
- Ejemplo:
- Fork > P/F/Fig1, Fig2, Fig3 > Broadcast
- Si un servidor se desconecta, envía:
- Server > P/K/ > Fork
- Si el servidor cae sin avisar, el Fork debe detectarlo con timeout.
- Si una figura no existe, se responde con error 404.
- Si el cliente cierra mal la conexión, se puede usar QUIT automático o timeout.

Futuras implementaciones a discutir

- Pedir 1 mitad
P/G/dragon :1
- Pedir 2 mitad
P/G/dragon :2

Bitácora

- La mayoría de los elementos que se registran son relacionados al FORK.
- Por ahora indica si la petición es con protocolo o si no.

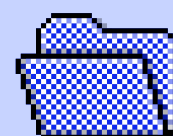


File System

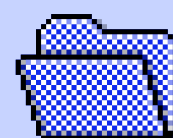
Implementación simplificada de un filesystem tipo Unix, basado en el uso de inodos y administración de bloques sobre un disco virtual representado por un archivo binario.

El sistema utiliza un archivo llamado “lego_data.bin” como disco, el cual está organizado en bloques de tamaño fijo de 512 bytes.

Todas las operaciones de almacenamiento, lectura y escritura se realizan a nivel de bloques, simulando el comportamiento de un sistema de archivos real.



Superbloque



Bitmap de bloques

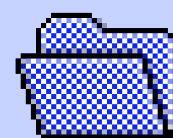
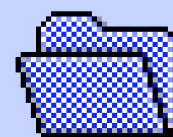


Tabla de inodos

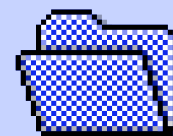
Punteros directos para archivos pequeños

Un puntero indirecto simple

Un puntero doble indirecto



Zona de datos

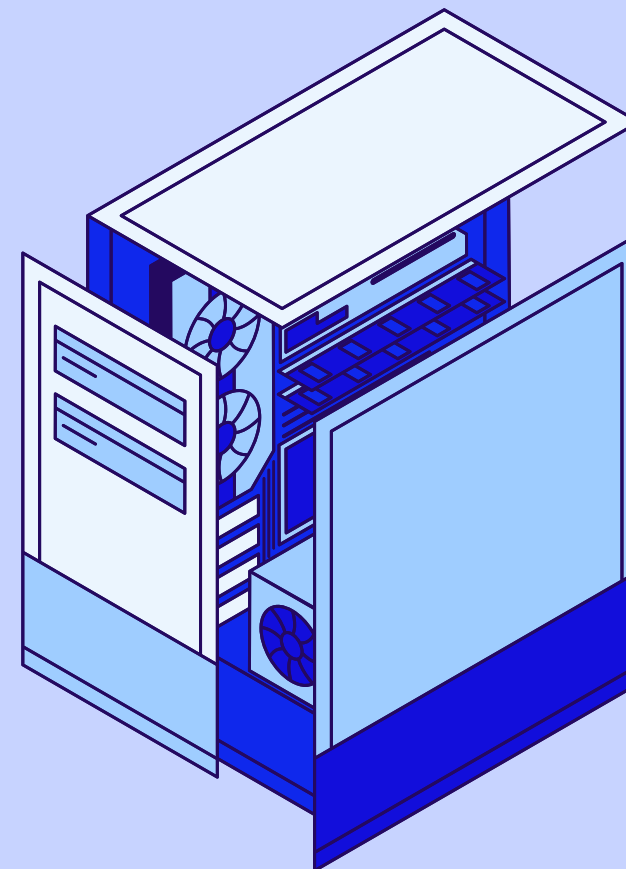


Directorios



Simulación: Protocolo

- **Cliente** — pide la lista de figuras o solicita una figura específica
- **Intermediario** — recibe la petición, la enruta al servidor, y si no se encuentra, hace fallback a otro intermediario
- **Intermediario 2** — actúa como una segunda mesa con su propio inventario de respaldo
- **Servidor** — busca la figura en el sistema de archivos principal y responde.



Se mantiene:

- Concurrencia, los datos y la estructura de hilos son exactamente iguales.

Se cambió:

- Toda la comunicación se adapta al nuevo protocolo



Breve intro a NachOS

En este proyecto podemos imaginar tres capas trabajando juntas:

Programa de usuario → Stub en ensamblador → Kernel de NachOS.

El programa legoclient corre como un programa normal de usuario. Pero cuando necesita hacer algo privilegiado, como crear un socket o enviar datos por red, no lo puede hacer directamente, entonces entra la segunda capa: los stubs en ensamblador MIPS.

En el archivo **start.s**, cada syscall tiene un pequeño stub. Ese stub es muy simple: lo único que hace es cargar en un registro un número que identifica cuál syscall se quiere ejecutar, y después ejecuta la instrucción syscall.

Cuando la CPU simulada ejecuta esa instrucción, ocurre algo parecido a una interrupción. El simulador MIPS detecta la syscall y genera una excepción. Esa excepción llega al kernel de NachOS, específicamente al archivo **exception.cc**.

Ahí el kernel revisa el registro donde venía guardado el número de syscall, identifica qué operación pidió el programa, y ejecuta la función correspondiente dentro del kernel.

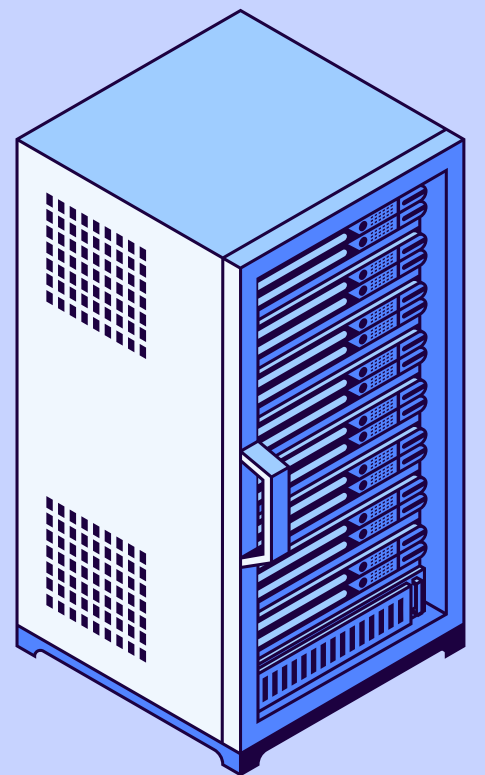
Cuando termina, el resultado se escribe nuevamente en un registro de la máquina simulada, y el programa continúa ejecutándose como si hubiera llamado una función normal.





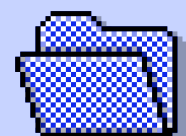
Syscalls

- **Socket()**: Aquí el programa le está diciendo al kernel que quiere crear un socket TCP, pero el kernel de NachOS no implementa TCP desde cero, lo que hace es apoyarse en Linux. Entonces llama al **socket()** real de Linux, obtiene un file descriptor Unix, y lo guarda en una tabla interna.
- **Connect()**: El kernel primero tiene que copiar ese **string** usando **machine** → **ReadMem**, luego arma una estructura **sockaddr_in** normal de Linux con la IP y el puerto, busca el file descriptor Unix real asociado al socket, y finalmente llama a **connect()** de Linux.
- **Write()**: El kernel copia los bytes desde la memoria simulada administrada por NachOS del programa, hacia un buffer real del kernel. Después revisa el descriptor: si corresponde a un socket, simplemente llama a **write()** de Linux sobre ese socket.
- **Read()**: El kernel llama a **read()** de Linux para recibir datos del servidor. Los datos llegan primero a un buffer interno del kernel, y después NachOS los copia hacia la memoria del programa usando **machine** → **WriteMem**. Finalmente retorna cuántos bytes llegaron. Si retorna 0 o un valor negativo, el programa interpreta que el servidor cerró la conexión o que ocurrió un error.



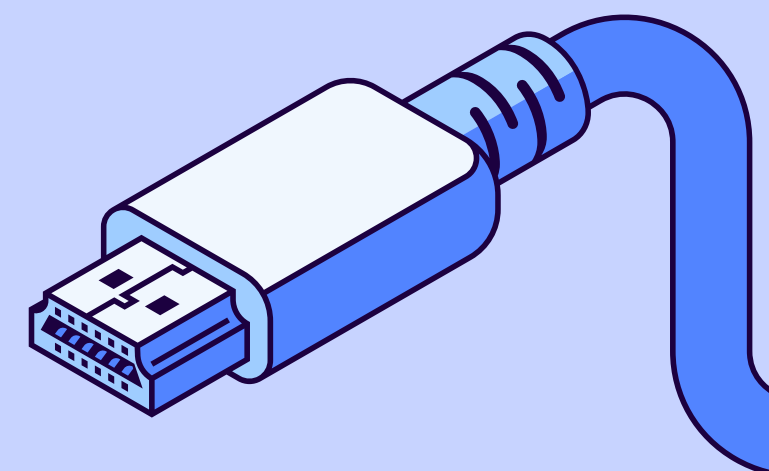


Syscalls

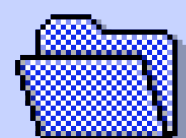


Close(sock)

- Busca el descriptor Unix real asociado al socket.
- Llama a close() de Linux.
- Limpia la entrada de su tabla interna.

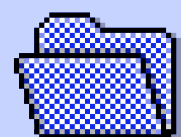


Consola



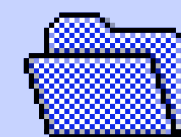
Write(ConsoleOutput)

- Imprime el texto en la terminal real usando printf()



Read(ConsoleInput)

- Lee caracteres directamente desde el teclado real usando

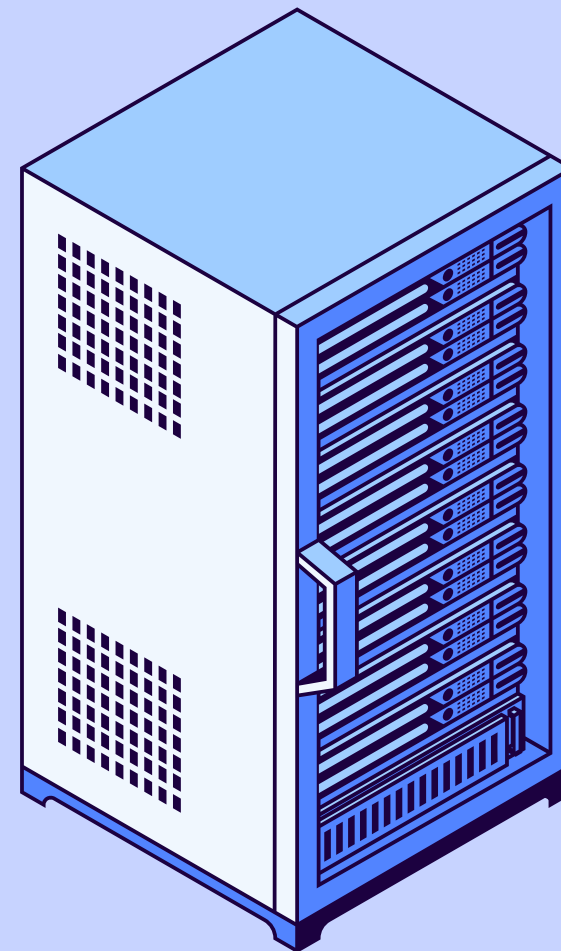


Exit()

- El kernel marca el proceso como terminado.
- Libera recursos
- Finaliza la ejecución.



SERVIDOR



- Se recibe la solicitud HTTP o la solicitud del protocolo intranet
- Si es necesario limpia el contenido, si no utiliza el formato HTML



os.ecci.ucr.ac.cr





Diseño y restricciones

- Al correr en NachOS no existe libc: sin printf, sin malloc, sin strlen
- El stack es pequeño → todos los buffers son variables globales
- Funciones auxiliares propias para suplir la falta de libc:
 - ***mystrlen*** → reemplaza strlen
 - ***mystrcat*** → reemplaza strcat, se usa para construir mensajes y paths
 - ***print*** → reemplaza printf usando la syscall Write a ConsoleOutput
 - ***readline*** → reemplaza fgets leyendo carácter por carácter desde ConsoleInput



Funcionamiento general

- Cada petición sigue el mismo ciclo:
Socket() → **Connect()** → **Write()** → **loop Read()** → **Close()**
- **doProtocolRequest:** envía el mensaje directo (ej: P/R/dir, P/G/fish:1) y acumula la respuesta hasta que el servidor cierra la conexión
- **doRequest:** igual, pero construye primero un HTTP request completo con el header User-Agent: nachos para que el servidor responda en texto plano. Luego localiza el separador entre headers y body, e imprime solo el contenido útil





Flujo – Prot. Intragrupal

- Envía P/R/dir → recibe la lista de figuras disponibles
- El usuario elige una figura y si quiere la parte 1, parte 2, o la figura completa
- Envía por ejemplo P/G/fish:1 → recibe P/D/2|plate 2x2 blue,1|slope 2x2 white
- Una sola petición puede devolver la figura completa



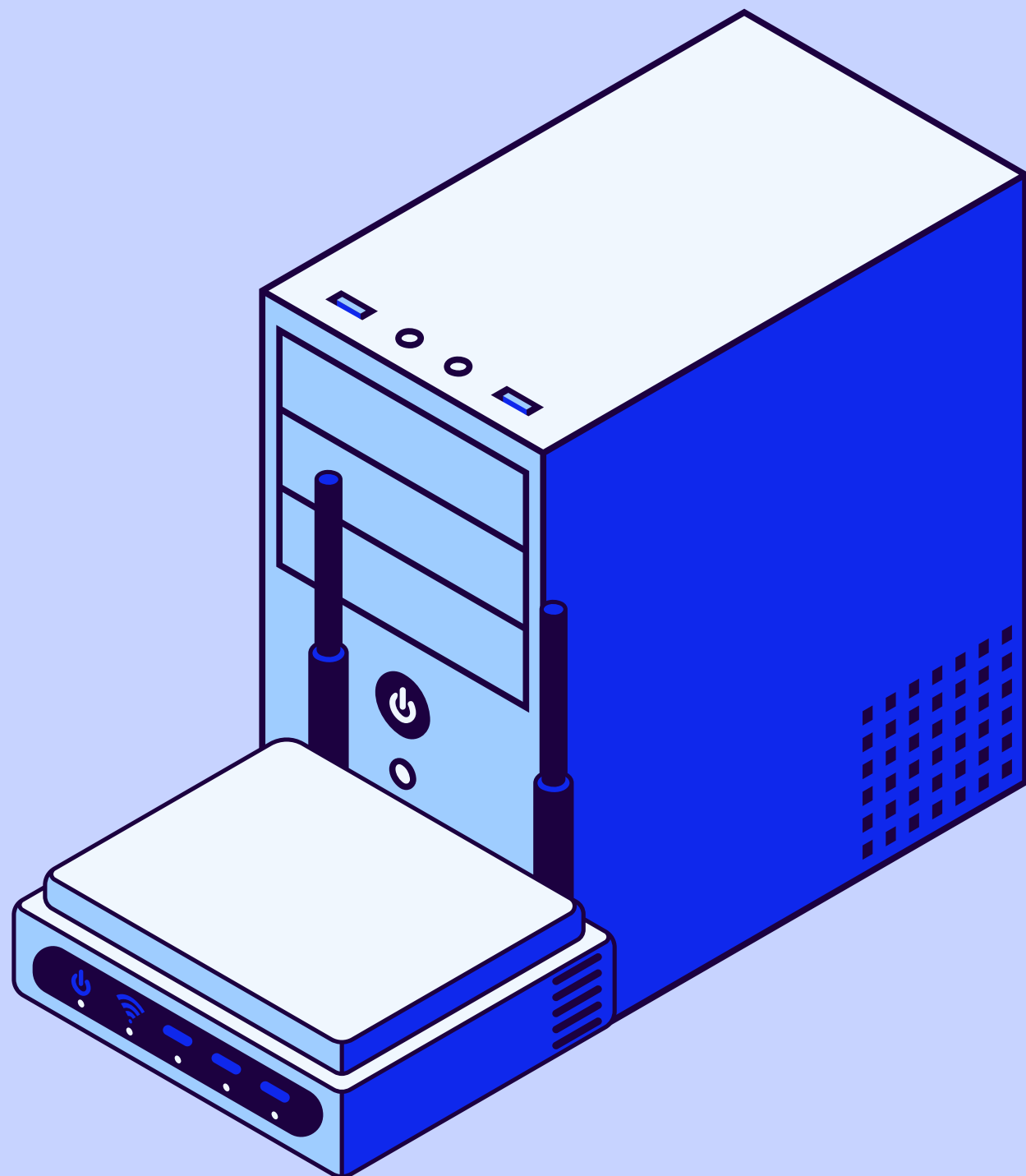
Flujo – HTTP

- Envía GET /lego/index.php con el header User-Agent: nachos
- El servidor detecta ese header y responde con texto plano en lugar de HTML
- El cliente ubica el inicio del body buscando `\r\n\r\n` en la respuesta
- El usuario elige figura y parte (1 o 2) → se consulta /lego/list.php?figure=X&part=Y

Casos de prueba (servidor)



Caso de prueba	Entrada / Condición	Resultado esperado	Estado
Listado de figuras por protocolo	Ejecutar el cliente con el servidor activo	Se envía P/R/dir y se muestran las figuras disponibles por protocolo	Manejado correctamente
Selección válida de figura y primera mitad por protocolo	Elegir una figura válida y opción 1	Se envía P/G/figura:1 y se muestra la primera mitad de la figura	Manejado correctamente
Selección válida de figura completa por protocolo	Elegir una figura válida y opción 3	Se envía P/G/figura y se muestra la figura completa	Manejado correctamente
Opción inválida en protocolo	Ingresar una opción diferente de 1, 2 o 3	El cliente muestra Opcion invalida. Use 1, 2 o 3 y termina sin hacer una solicitud incorrecta	Manejado correctamente
Listado de figuras por HTTP	Continuar a la sección HTTP del cliente	Se envía GET /lego/index.php y se muestran las figuras disponibles	Manejado correctamente
Selección válida de figura y parte por HTTP	Elegir una figura válida y parte 1 o 2	Se envía GET /lego/list.php?figure=figura&part=parte y se muestra la lista de piezas	Manejado correctamente
Parte inválida en HTTP	Ingresar una parte diferente de 1 o 2	El cliente muestra Parte invalida, use 1 o 2 y finaliza	Manejado correctamente
Servidor sin respuesta	El servidor no responde o no devuelve datos	El cliente muestra un mensaje indicando que no recibió figuras o respuesta del servidor	Manejado correctamente



Configuración Switches L2 / L3

- Creación de la VLAN 220 (vlan 220) y asignación del nombre ISLA2.
- Configuración de la IP 172.16.123.34/28 en interface vlan 220 y definición del gateway 172.16.123.33.
- Configuración de los puertos fa0/1-2 en modo access para la VLAN 220 y del puerto fa0/24 en modo trunk hacia el L3.
- Configuración del servicio DHCP para la red 172.16.123.32/28, excluyendo las IPs 172.16.123.33 y 172.16.123.34.



Etapa 2 - Equipo 3

Muchas gracias